

6.1. Introduction to Runtime View

`{#runtime-view}`

This section describes how the building blocks (Django apps and their components) collaborate at runtime to fulfill key system functionalities. We will illustrate these interactions using sequence diagrams for a few representative scenarios. These diagrams show the order of operations and messages passed between components.

6.2. Key Scenarios

{#_6_2_key_scenarios}

We will explore the following key runtime scenarios:

1. Jobseeker searches for vacancies and views details.
2. Publisher posts a new vacancy (simplified).
3. Daily job alerts are sent (background process).

6.2.1. Scenario 1: Jobseeker Searches for Vacancies and Views Details

{#_6_2_1_scenario_1_jobseeker_searches_for_vacancies_and_views_details}

Description: A jobseeker uses the platform to search for teaching vacancies based on criteria like keyword, location, subject, etc. They then view the details of a specific vacancy from the search results.

```
@startuml
!theme materia
actor Jobseeker
participant "Browser" as WebBrowser
participant "Django Web App\n(vacancies.views)" as DjangoViews
participant "SearchForm\n(jobseekers.forms)" as SearchForm
participant "VacancySearchService\n(search.services)" as SearchService
participant "VacancyModel\n(vacancies.models)" as VacancyMdl
participant "Pagy" as Paginator
database "PostgreSQL DB" as DB
```

```
group Search for Vacancies
Ê Jobseeker -> WebBrowser : Enters search criteria (keyword, location, etc.), submits form
Ê WebBrowser -> DjangoViews : HTTP GET /vacancies/?keyword=maths&location=london
Ê DjangoViews -> SearchForm : Validate and clean criteria(request.GET)
Ê SearchForm --> DjangoViews : Validated search parameters
Ê DjangoViews -> SearchService : perform_search(params)
Ê SearchService -> VacancyMdl : build_queryset(params)
Ê VacancyMdl -> DB : SELECT * FROM vacancies_vacancy WHERE searchable_content @@ to_tsquery(...) AND location ...
Ê DB --> VacancyMdl : Raw vacancy data
Ê VacancyMdl --> SearchService : Vacancy queryset
Ê SearchService -> Paginator : paginate(queryset)
Ê Paginator --> SearchService : Paginated queryset & pagy object
Ê SearchService --> DjangoViews : Search results (paginated vacancies, pagy object)
Ê DjangoViews -> WebBrowser : HTTP Response (HTML page with search results and pagination)
Ê WebBrowser --> Jobseeker : Displays search results
end
```

```
group View Vacancy Details
Ê Jobseeker -> WebBrowser : Clicks on a vacancy link
Ê WebBrowser -> DjangoViews : HTTP GET /vacancies/{vacancy_slug}/
Ê DjangoViews -> VacancyMdl : find_by_slug(vacancy_slug)
```

```

Ê VacancyMdl -> DB : SELECT * FROM vacancies_vacancy WHERE slug = ...
Ê DB --> VacancyMdl : Vacancy data
Ê VacancyMdl --> DjangoViews : Vacancy object
Ê DjangoViews -> SearchService : find_similar_jobs(vacancy)
Ê SearchService -> DB : (Queries for similar vacancies)
Ê DB --> SearchService : Similar vacancy data
Ê SearchService --> DjangoViews : Similar job list
Ê DjangoViews -> WebBrowser : HTTP Response (HTML page with vacancy details and
similar jobs)
Ê WebBrowser --> Jobseeker : Displays vacancy details
end
@enduml

```

Explanation of Sequence (Jobseeker Searches and Views):

1. Search Submission: The Jobseeker enters search terms (e.g., keyword, location) into the search form in their browser and submits it.
2. HTTP Request: The browser sends an HTTP GET request to the Django application's vacancy index view, including search parameters in the query string.
3. Form Processing: The Django view instantiates a **SearchForm** with the request data. The form validates and cleans the input parameters.
4. Search Service: The view calls a **VacancySearchService** (or similar service/manager method) with the validated parameters.
5. Query Building: The search service interacts with the **VacancyModel** to build a database query based on the criteria. This involves filtering by keywords (using PostgreSQL's full-text search on **searchable_content**), location (potentially using PostGIS functions on **geopoint** or **geopoint_placeholder** initially), and other facets.
6. Database Query: The Django ORM executes the SQL query against the PostgreSQL database.
7. Results Returned: The database returns the raw data for matching vacancies.
8. Pagination: The search service uses a pagination library (Pagy) to paginate the results.
9. Response Generation: The Django view receives the paginated list of vacancy objects and renders an HTML template, displaying the search results.
10. Display Results: The browser renders the HTML page for the Jobseeker.
11. View Vacancy: The Jobseeker clicks on a specific vacancy link from the search results.
12. HTTP Request (Vacancy Detail): The browser sends an HTTP GET request to the vacancy detail view, typically using the vacancy's slug.
13. Fetch Vacancy: The view retrieves the specific **Vacancy** object from the database via its model, using the slug.
14. Fetch Similar Jobs: The view might call the **SearchService** again to find and display similar job listings based on the current vacancy's attributes.
15. Response Generation (Vacancy Detail): The view renders an HTML template with the full details of the selected vacancy and any similar jobs.

16. Display Vacancy: The browser displays the detailed vacancy page to the Jobseeker.

6.2.2. Scenario 2: Publisher Posts a New Vacancy (Simplified)

{#_6_2_2_scenario_2_publisher_posts_a_new_vacancy_simplified}

Description: A publisher (school staff) logs in and creates a new job vacancy through a multi-step form process. This is a simplified version focusing on key component interactions rather than every step of a potentially complex wizard.

```
@startuml
!theme materia
actor Publisher
participant "Browser" as WebBrowser
participant "Django Web App\n(publishers.vacancies.views)" as DjangoViews
participant "VacancyForm\n(publishers.forms)" as VacancyForm
participant "VacancyModel\n(vacancies.models)" as VacancyMdl
participant "OrganisationModel\n(organisations.models)" as OrganisationMdl
database "PostgreSQL DB" as DB
participant "Celery Worker" as Celery
queue "Redis (Task Queue)" as TaskQueue
```

```
Publisher -> WebBrowser : Navigates to "Create Vacancy"
WebBrowser -> DjangoViews : HTTP GET /publishers/vacancies/new/ (or wizard start)
DjangoViews -> WebBrowser : HTTP Response (HTML form for first step)
```

```
loop Multi-step Vacancy Form
  Ê Publisher -> WebBrowser : Fills in form section, submits
  Ê WebBrowser -> DjangoViews : HTTP POST /publishers/vacancies/wizard/{step_id}/
  Ê DjangoViews -> VacancyForm : Validate and process_step(request.POST,
current_vacancy_draft)
  Ê alt Form step is valid
    Ê VacancyForm -> VacancyMdl : Update/save draft vacancy attributes
    Ê VacancyMdl -> DB : INSERT/UPDATE vacancies_vacancy ...
    Ê DB --> VacancyMdl : Success
    Ê VacancyMdl --> VacancyForm : Updated vacancy draft
    Ê VacancyForm --> DjangoViews : Step processed successfully
    Ê DjangoViews -> WebBrowser : HTTP Response (HTML for next step OR review page)
  Ê else Form step is invalid
    Ê VacancyForm --> DjangoViews : Validation errors
    Ê DjangoViews -> WebBrowser : HTTP Response (HTML form with errors)
  Ê end
end
```

```

group Final Review and Publish
Ê Publisher -> WebBrowser : Reviews complete draft, clicks "Publish"
Ê WebBrowser -> DjangoViews : HTTP POST /publishers/vacancies/{vacancy_id}/publish/
Ê DjangoViews -> VacancyMdl : Retrieve vacancy_draft
Ê VacancyMdl -> DB : SELECT * FROM vacancies_vacancy WHERE id = ...
Ê DB --> VacancyMdl : Vacancy data
Ê DjangoViews -> VacancyForm : Final validation (if any specific publish checks)
Ê alt Publish is valid
Ê   VacancyForm --> DjangoViews : Valid
Ê   DjangoViews -> VacancyMdl : vacancy.status = PUBLISHED, vacancy.publish_on = today
Ê   VacancyMdl -> DB : UPDATE vacancies_vacancy SET status='PUBLISHED', ...
Ê   DB --> VacancyMdl : Success
Ê   DjangoViews -> TaskQueue : Enqueue task (e.g., update_search_index,
notify_services)
Ê   TaskQueue -> Celery : Dispatches task
Ê   Celery -> VacancyMdl : (Potentially re-fetches vacancy)
Ê   Celery -> DB : (e.g., Updates SearchVectorField if handled by Celery)
Ê   Celery --> TaskQueue : Task complete
Ê   DjangoViews -> WebBrowser : HTTP Response (HTML confirmation page, "Vacancy
Published")
Ê   WebBrowser --> Publisher : Displays confirmation
Ê else Publish is invalid (e.g., missing required fields identified at final review)
Ê   DjangoViews -> WebBrowser : HTTP Response (HTML review page with errors)
Ê end
end
@enduml

```

Explanation of Sequence (Publisher Posts Vacancy):

1. Initiate Creation: The Publisher navigates to the "Create Vacancy" section.
2. First Step Form: The Django view for the vacancy creation wizard renders the first step of the form.
3. Multi-Step Form Submission:
 - ! The Publisher fills in a section of the vacancy form and submits it.
 - ! The browser POSTs the data to the corresponding wizard step view.
 - ! The view uses a **VacancyForm** (or a step-specific form) to validate the submitted data and associate it with the current draft vacancy (which might be created on the first step or retrieved from the session/database).
 - ! If the step's data is valid, the attributes are saved to the **VacancyModel** (often with a **status** of DRAFT). The database is updated.
 - ! The view then redirects the Publisher to the next step in the wizard or to a review page if all steps are complete.
 - ! If invalid, the form with errors is re-rendered.
4. Final Review and Publish:
 - ! After completing all steps, the Publisher reviews the full draft vacancy.

- ! Upon clicking \"Publish,\" the browser POSTs to a publish action URL.
- ! The view retrieves the draft vacancy.
- ! Final validation checks (if any specific to publishing) are performed.
- ! If valid, the vacancy's **status** is updated to PUBLISHED, and **publish_on** date is set. The **VacancyModel** is saved to the database.
- ! Asynchronous Tasks: The view enqueues background tasks using Celery (via Redis as a broker). These tasks might include:
 - ! Updating the **searchable_content** field for the vacancy.
 - ! Notifying external services (e.g., DWP Find a Job, Google Indexing).
- ! The Celery worker picks up these tasks and executes them, potentially interacting with the database or external APIs.
- ! The view responds to the Publisher with a confirmation page.

6.2.3. Scenario 3: Daily Job Alerts Sent (Background Process)

{#_6_2_3_scenario_3_daily_job_alerts_sent_background_process}

Description: The system automatically sends out daily email notifications to jobseekers who have active subscriptions (job alerts) for vacancies matching their criteria that were posted recently.

```
@startuml
!theme materia
participant "Scheduler (Celery Beat)" as Scheduler
participant "Celery Worker\n(subscriptions.tasks)" as CeleryWorker
participant "SubscriptionModel\n(jobseekers.models)" as SubscriptionMdl
participant "VacancyModel\n(vacancies.models)" as VacancyMdl
database "PostgreSQL DB" as DB
participant "GOV.UK Notify Client" as NotifyClient
```

```
Scheduler -> CeleryWorker : Triggers daily_job_alerts_task
CeleryWorker -> SubscriptionMdl : Get active daily subscriptions
SubscriptionMdl -> DB : SELECT * FROM jobseekers_subscription WHERE active=true AND frequency='DAILY'
DB --> SubscriptionMdl : Returns list of subscriptions
SubscriptionMdl --> CeleryWorker : List of active daily subscriptions
```

```
loop For each Subscription
  Ê CeleryWorker -> VacancyMdl : Find new vacancies matching subscription.search_criteria
  Ê VacancyMdl -> DB : SELECT * FROM vacancies_vacancy WHERE publish_on >= (yesterday) AND status='PUBLISHED' AND (matches criteria...)
  Ê DB --> VacancyMdl : List of matching vacancies
  Ê VacancyMdl --> CeleryWorker : List of new matching vacancies
```

```
Ê alt If matching vacancies found
  Ê CeleryWorker -> NotifyClient : Send_email(to=subscription.email, template_id=..., personalization={vacancies_list})
  Ê NotifyClient -> "GOV.UK Notify API" : API Call
  Ê "GOV.UK Notify API" --> NotifyClient : Success/Failure
  Ê NotifyClient --> CeleryWorker : Email sent status
  Ê CeleryWorker -> SubscriptionMdl : Log alert sent (e.g., create AlertRun record)
  Ê SubscriptionMdl -> DB : INSERT INTO jobseekers_alertrun ...
  Ê DB --> SubscriptionMdl : Success
Ê end
```

```
end
```

```
CeleryWorker -> CeleryWorker : Task complete  
@enduml
```

Explanation of Sequence (Daily Job Alerts):

1. **Scheduled Trigger:** A scheduler (like Celery Beat) triggers a predefined periodic task (e.g., `send_daily_job_alerts_task`) that runs once a day.
2. **Task Execution:** A Celery worker picks up this task.
3. **Fetch Subscriptions:** The worker queries the `SubscriptionModel` to retrieve all active subscriptions with a `"daily"` frequency.
4. **Database Query (Subscriptions):** The ORM fetches these subscriptions from the PostgreSQL database.
5. **Process Each Subscription:** For each active daily subscription:
 - ! **Find Matching Vacancies:** The worker queries the `VacancyModel` to find newly published vacancies (e.g., `publish_on` date is yesterday or today, `status` is PUBLISHED) that match the `search_criteria` stored in the current subscription. This involves complex filtering logic similar to the job search scenario.
 - ! **Database Query (Vacancies):** The ORM executes the query for matching vacancies.
 - ! **Email Generation:** If new matching vacancies are found for the subscription:
 - ! The worker prepares the email content, typically personalizing a pre-defined template with the list of vacancies.
 - ! It uses the `NotifyClient` (a wrapper for the GOV.UK Notify API client) to send the email to the `subscription.email`.
 - ! The `NotifyClient` makes an API call to the GOV.UK Notify service.
 - ! **Log Alert:** The worker may create an `AlertRun` record (or update a similar tracking mechanism) to log that an alert was sent for this subscription on this day.
6. **Task Completion:** After processing all relevant subscriptions, the Celery task finishes.

These scenarios provide a snapshot of the dynamic behavior of the system, illustrating how different components collaborate to achieve user goals and perform automated tasks.